MuseRelay AWS Engagement Bridge

One AI agent for WhatsApp and web chat, with tickets that follow the customer across channels — built on AWS End User Messaging Social and Amazon Bedrock

Entrant: Screen Art SL (Spain) · **Event:** AWS CDS Agentic AI Partner Hackathon 2026 · **Date:** 28 October 2026
Code repository (open source, Apache 2.0): https://github.com/screenartsl/aws-engagement-bridge
Live demo: https://demo.screenart.es (fictitious company "Screen Art Connect") · **Platform:** https://muserelay.com (SaaS in beta)

1. Summary

MuseRelay is a customer-service SaaS (beta) with a unified inbox, AI agents with business tools, ticketing, calendars and white-label resellers. For this hackathon we built the **AWS Engagement Bridge**, a new module that carries WhatsApp traffic through **AWS End User Messaging Social** into MuseRelay, and back, using the AWS SDK at runtime.

On top of it, one MuseRelay AI agent running **Claude Haiku 4.5 on Amazon Bedrock** answers both WhatsApp (through AWS) and a web chatbot. A support ticket opened on one channel can be continued on the other after a server-side one-time-code check, without duplicating the ticket. Operators supervise from the same inbox and ticket page.

2. Architecture

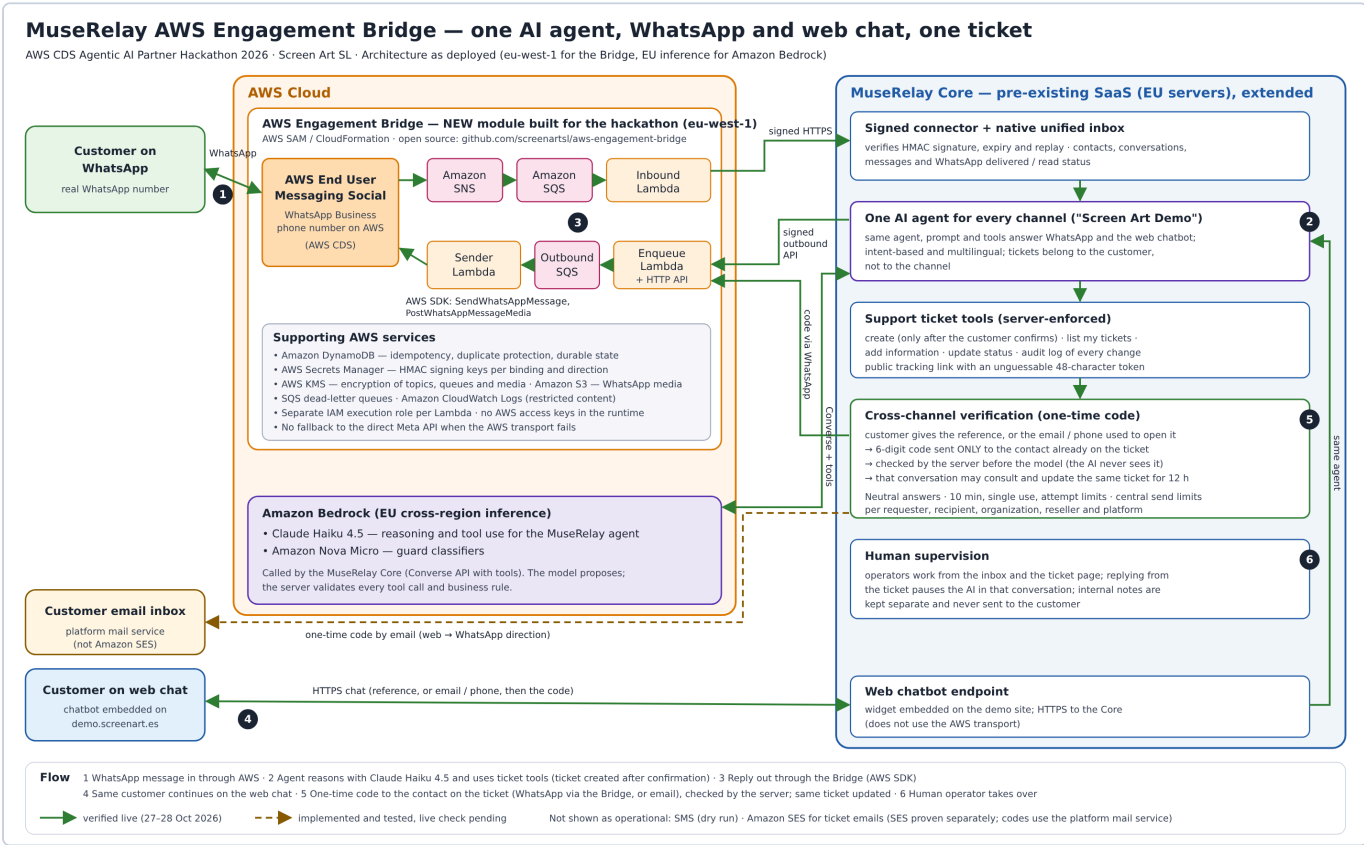


Figure 1 — Architecture as deployed. Solid green arrows: verified live on 27–28 October 2026. Dashed: implemented and covered by automated tests, live check pending.

Message flow

1. **Inbound:** customer WhatsApp → AWS End User Messaging Social → Amazon SNS → Amazon SQS → Inbound Lambda → signed HTTPS request → MuseRelay connector → native inbox (contact, conversation, message).
2. **Reasoning:** the MuseRelay agent calls Amazon Bedrock (Claude Haiku 4.5, Converse API with tool definitions). The model proposes tool calls; MuseRelay executes them and enforces business rules (for example, a ticket is only created after the customer confirms).
3. **Outbound:** MuseRelay → signed outbound API (HTTP API) → Enqueue Lambda → outbound SQS → Sender Lambda → AWS SDK `SendWhatsAppMessage` (and `PostWhatsAppMessageMedia` for files) → customer. Delivered and read statuses come back on the inbound path.
4. **Channel change:** the same customer opens the web chatbot (same agent) and gives the ticket reference — or, if they do not remember it, the email or phone used to open it.
5. **Verification:** a 6-digit one-time code is sent only to a contact already on the ticket (its WhatsApp, through the Bridge, or its email). The server checks the code before the model is called; the conversation is then authorised for that ticket and the customer can read its status and add information to the same ticket.
6. **Human supervision:** an operator replies from the ticket page; the AI pauses in that conversation. Internal notes are stored separately and never sent.

3. Scope: what is new and what already existed

Built during the hackathon (from 24 October 2026)	Pre-existing MuseRelay platform
AWS Engagement Bridge module (open-source repository): AWS SAM / CloudFormation template; Inbound, Enqueue, Sender and Notify Lambdas (TypeScript); AWS SDK WhatsApp sender and media handling; signed HTTPS contract (HMAC per binding and direction, expiry, replay protection); DynamoDB idempotency; Laravel connector that writes into the native inbox; WhatsApp status synchronisation. **Core changes for the demo journey** (MuseRelay, proprietary): cross-channel ticket continuity with one-time codes (by reference, or by email / phone); central sending limits for every verification code on the platform; the code is checked by the server before the model; the AI pauses when an operator replies from a ticket; separate internal notes; customer media and delivery status on the ticket page.	SaaS application, organisations, workspaces and white-label resellers; unified inbox and CRM; AI agent framework with tool calling and Amazon Bedrock integration; ticketing, calendar and booking tools; web chatbot widget; other channels (email, Telegram, Instagram, Messenger) with different maturity levels. The pre-existing platform is **not** presented as hackathon work. The MuseRelay Core is proprietary and is not included in the repository; the repository contains the Bridge module and its connector.

4. AWS services used at runtime

Service	How it is used

AWS End User Messaging Social	WhatsApp Business number on AWS. Runtime calls through `@aws-sdk/client-socialmessaging`: `SendWhatsAppMessage`, `PostWhatsAppMessageMedia`, `GetWhatsAppMessageMedia`, `ListWhatsAppMessageTemplates`, `GetWhatsAppMessageTemplate` (`src/social.ts`, `src/media.ts`).
Amazon Bedrock (EU inference)	Claude Haiku 4.5 for agent reasoning and tool use; Amazon Nova Micro for guard classifiers. Called by the MuseRelay Core.
AWS Lambda + Amazon API Gateway (HTTP API)	Inbound, Enqueue, Sender and Notify functions; signed outbound API.
Amazon SNS, Amazon SQS	Inbound event topic; inbound and outbound queues with dead-letter queues.
Amazon DynamoDB	Idempotency, duplicate protection and durable processing state.
Amazon S3, AWS KMS	WhatsApp media storage; encryption of topics, queues and media.
AWS Secrets Manager, IAM	HMAC signing keys; one execution role per Lambda; no AWS access keys in the messaging runtime.
Amazon CloudWatch Logs	Function logs with restricted content.
Amazon SES / End User Messaging SMS	Notify Lambda: an SES email was verified once; SMS runs in dry-run mode only. **Neither is part of the demonstrated journey**: ticket codes by email use MuseRelay's platform mail service.

5. Security, privacy and human control

Transport (Bridge)

- Independent HMAC keys per binding and direction, with signature, expiry and replay checks; invalid signatures and duplicates rejected (verified live).
- Separate IAM roles per Lambda; AWS-side secrets in Secrets Manager; no Meta access tokens stored in MuseRelay for AWS channels; no fallback to the direct Meta API.
- Bindings map each WhatsApp number to one MuseRelay organisation and channel.

Agent and tools (Core)

- The model proposes; the server decides. Ticket creation requires the customer's confirmation; status changes and access checks run in the tool code, not in the prompt.
- Ticket lookups are limited to the agent's own ticket inbox and workspace (organisation and reseller isolation).
- Human-in-the-loop: replying from the ticket page pauses the AI in that conversation; internal notes never reach the customer; every change is written to an audit log.

Cross-channel verification (one-time codes)

- The code goes only to a contact already registered on the ticket — never to a detail typed in the chat. Knowing a reference, email or phone does not grant access.
- The same neutral answer is returned whether or not a ticket exists, with a minimum response time.
- Codes: 6 digits, stored only as a hash, valid 10 minutes, single use; 3 attempts per conversation (15-minute lock) and 5 failures in total across sessions before the code is invalidated. A valid code is reused instead of re-sent.
- The server checks the code **before** calling the model; the model only sees "••••••" and the result. Codes are masked in stored messages, in all application logs and in tool-call records.
- Central sending limits for every code sent by the platform (shared Redis counters, atomic): per requester (public IP or channel identity) 5 per 10 minutes and 20 per day; enumeration detection (a 4th distinct reference, email or phone in one hour blocks the origin for 60 minutes and alerts the team); per recipient 3 per hour and 6 per day across organisations and channels; per organisation 30, per reseller 100 and platform-wide 300 per hour.
- Abuse is written to a security log (without codes) and alerts are emailed to the platform administrator.

> Data location: the Bridge runs in AWS eu-west-1 and Bedrock is used through EU inference. Using EU infrastructure does not by itself guarantee GDPR compliance; contractual and organisational measures are also required.

6. Verified live results

Capability	Status
WhatsApp text received through AWS; contact and conversation created in the inbox	**Verified**
Human reply sent with `SendWhatsAppMessage`; delivered and read statuses	**Verified**

Invalid signature and duplicate rejection	**Verified**
AI reply through the Bridge (Claude Haiku 4.5 on Amazon Bedrock)	**Verified**
Ticket actions by the AI over WhatsApp through AWS: ticket TK-2026-00001 created after confirmation, information added, status changed (27 Oct 2026)	**Verified**
Inbound photo and voice note fetched (`GetWhatsAppMessageMedia`) and stored; file sent from the inbox (`PostWhatsAppMessageMedia`)	**Verified**
Web chatbot answered by the same agent (28 Oct 2026)	**Verified**
WhatsApp → web continuity: one-time code delivered through AWS to the ticket's WhatsApp and accepted in the web chat (28 Oct 2026). The first live run showed an empty reply after a correct code; fixed the same day (server checks the code before the model)	**Code path verified; full re-run pending**
Web → WhatsApp continuity (code to the ticket's email); recovery without the reference (email or phone)	**Implemented, tested; live check pending**
WhatsApp template send; inbound PDF; calendar action through the Bridge	**Implemented; live check pending**
Confirmation email through Amazon SES (Notify Lambda, one email to a verified test recipient)	**Verified**
SMS through End User Messaging SMS	**Dry run only — not claimed**

Automated tests: the Bridge repository runs its own suite without AWS credentials. The MuseRelay Core changes are covered by unit and feature tests (multi-session, multi-IP and multi-organisation abuse scenarios, cross-channel continuity), executed against an isolated test database before each deployment.

7. How to test

A. Repository (no AWS credentials needed)

```
git clone https://github.com/screenartsl/aws-engagement-bridge
cd aws-engagement-bridge
npm ci --ignore-scripts
npm test
npm run bundle
```

Requirements: Node.js 22 (and PHP 8.1+ for the connector tests in `docs/STAGING.md`). The AWS SDK call sites are in `src/social.ts` (Sender Lambda) and `src/media.ts`. Deployment: `template.yaml` and `docs/CLOUDFORMATION-DEPLOYMENT.md`; evidence and status of every capability (verified live / implemented with tests / not claimed): `docs/EVALUATION.md`.

B. Live demo

1. Open https://demo.screenart.es and use the chat in the corner. Use fictitious data, but an email address you can read.

2. Try: *“My internet connection is down. I need help.”* The agent asks for confirmation, a name and an email, and gives a reference such as TK-2026-00002.
3. Open the page in a private window (a new conversation) and try: *“I already have a support request, but I don't remember the ticket number.”* Give the email you used; type the code you receive; the agent shows your requests and lets you add information.
4. The WhatsApp side of the journey (through AWS End User Messaging Social) is shown in the demo video.

> Abuse protection also applies to testers: from one IP, asking for codes for more than 3 different references, emails or phones within an hour blocks that origin for 60 minutes.

8. Limitations and next steps

- WhatsApp codes outside the 24-hour customer-service window need an approved authentication template; today the code goes to the ticket's email in that case.
- Planned: SMS through End User Messaging SMS (after sender ID registration), Amazon SES for ticket notifications, more agent tools over AWS channels, and voice — the same agent answering by phone.
- Video, stickers and documents other than PDF are ignored on inbound WhatsApp.